

Part 2: Depth First Traversals

Preorder, Inorder, Postorder

What traversal means

Traversal means visiting every node exactly once in some fixed order. That is all.

An array has one obvious order: left to right. A tree does not. From node 1 you can go to 2 or to 3, and there is no single natural answer. Different orders are useful for different jobs, so we standardise a few of them and give them names.

Depth First Search (DFS) means: go as deep as possible down one branch before coming back and trying the other branch. It is what a person does in a maze when they follow one corridor to its end before backtracking.

Breadth First Search (BFS) means: finish an entire level before moving to the next. That is Part 3.

DFS on a binary tree comes in three flavours. They differ in exactly one thing: **when you record the current node relative to the two recursive calls.**

```
// preorder
visit(node); go(left); go(right);

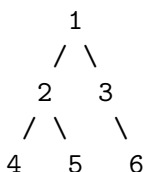
// inorder
go(left); visit(node); go(right);

// postorder
go(left); go(right); visit(node);
```

The names tell you where “root” sits: **pre** = root first, **in** = root in the middle, **post** = root last. In all three, left is always explored before right.

The one picture that explains all three

Imagine walking around the outside of the tree, keeping your left hand on the wall, starting and ending at the root.



During that walk you touch every node exactly **three** times:

1. when you arrive at it from above (going down into it)
2. when you come back up from its left subtree
3. when you come back up from its right subtree, just before leaving it for good

Now the three traversals are simply three choices of which touch counts:

Traversal	Record the node on which touch
Preorder	1st touch, on the way down
Inorder	2nd touch, between left and right
Postorder	3rd touch, on the way out

The results for our tree:

```
Preorder   : 1 2 4 5 3 6
Inorder    : 4 2 5 1 3 6
Postorder   : 4 5 2 6 3 1
```

Check those three lines against the picture until they feel obvious. Once this single picture is solid, iterative traversal and Morris traversal later become much less mysterious, because they are just different ways of remembering “which touch am I on”.

Preorder traversal

Meaning

Process the node, then everything on its left, then everything on its right. Root comes before its descendants, always.

Recursive algorithm

```
preorder(node):
    if node is empty: return
    record node
    preorder(node.left)
    preorder(node.right)
```

C++

```
void preorder(TreeNode* root, vector<int>& out) {
    if (root == NULL) return;
    out.push_back(root->val);
    preorder(root->left, out);
    preorder(root->right, out);
}
```

The LeetCode wrapper (problem 144):

```
vector<int> preorderTraversal(TreeNode* root) {
    vector<int> out;
    preorder(root, out);
    return out;
}
```

Note the `vector<int>&`. The ampersand means all calls share one vector. Drop it and every call gets its own copy, the pushes are thrown away, and you return an empty answer. This is one of the most common beginner bugs in tree code.

Dry run

call preorder(1)	record 1	out = [1]
call preorder(2)	record 2	out = [1,2]
call preorder(4)	record 4	out = [1,2,4]
preorder(NULL)	return	
preorder(NULL)	return	
return from 4		
call preorder(5)	record 5	out = [1,2,4,5]
two NULL calls, return		
return from 5		
return from 2		
call preorder(3)	record 3	out = [1,2,4,5,3]
preorder(NULL)	return	
call preorder(6)	record 6	out = [1,2,4,5,3,6]
return from 6		
return from 3		
done		out = [1,2,4,5,3,6]

As a call tree:

```
pre(1)
|-- pre(2)
|   |-- pre(4)
|   +-- pre(5)
+-- pre(3)
    +-- pre(6)
```

Read the node values in the order the calls **start**, and you get the preorder.

Complexity

Time $O(N)$: every node is entered once and does $O(1)$ work. Space $O(H)$ for the call stack, where H is the height. $O(\log N)$ if balanced, $O(N)$ if skewed. The output vector is a separate $O(N)$, which we usually do not count as extra space since it is the answer.

When preorder is the right choice

- **Copying or cloning a tree.** You must create the parent before you can attach children to it.
- **Serialising a tree** so it can be rebuilt later. Preorder plus null markers is enough to reconstruct a tree uniquely.
- **Building a tree from preorder plus inorder arrays** (LeetCode 105). The first element of preorder

is always the root.

- **Passing information downward**, for example “here is the path from the root to you” or “here is the sum so far”. Anything the parent knows and the child needs is naturally handed over on the way down. This is the shape of all root-to-leaf path problems in Part 9.

The rule of thumb: preorder is the traversal for **top-down** information flow.

Common mistakes

- Forgetting `&` on the output vector.
- Putting the `push_back` after the two calls “because it looks tidier”. That is postorder, and the output changes.
- Checking if `(root->left != NULL)` `preorder(root->left)` instead of guarding inside the function. It works, but it doubles the number of conditions you must get right, and it breaks immediately for an empty tree.

Inorder traversal

Meaning

Everything on the left, then the node, then everything on the right.

Recursive algorithm

```
inorder(node):
    if node is empty: return
    inorder(node.left)
    record node
    inorder(node.right)
```

C++

```
void inorder(TreeNode* root, vector<int>& out) {
    if (root == NULL) return;
    inorder(root->left, out);
    out.push_back(root->val);
    inorder(root->right, out);
}
```

Dry run

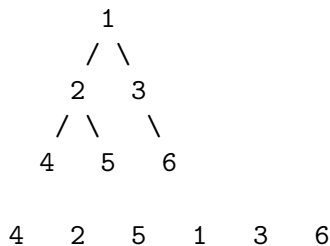
```
in(1)
  in(2)
    in(4)
      in(NULL) return
      record 4           out = [4]
      in(NULL) return
    record 2             out = [4,2]
  in(5)
    record 5             out = [4,2,5]
  record 1               out = [4,2,5,1]
```

```

in(3)
  in(NULL) return
  record 3          out = [4,2,5,1,3]
  in(6)
    record 6        out = [4,2,5,1,3,6]
done               out = [4,2,5,1,3,6]

```

A useful visual shortcut: drop every node straight down onto a horizontal line, keeping left nodes to the left. Reading that line from left to right gives the inorder sequence.



That is exactly the projection of the tree onto the x axis, and it is also the seed of the vertical order idea in Part 11.

Complexity

Time $O(N)$, space $O(H)$. Same as preorder.

When inorder is the right choice

- **Anything involving a BST.** The inorder traversal of a BST is sorted, ascending. This single fact solves a whole family of problems: validate BST (LeetCode 98), kth smallest element (230), minimum absolute difference (530), convert BST to sorted list, and more.
- **Finding the predecessor or successor of a node** in a BST.
- Recovering a tree when you are given the inorder array plus preorder or postorder.

Why is a BST's inorder sorted? By definition, everything in the left subtree is smaller than the node and everything in the right subtree is larger. Inorder prints exactly in that order: smaller stuff, node, larger stuff. Apply the same reasoning inside each subtree and the whole sequence is sorted.

Common mistakes

- Assuming inorder is sorted for a **general** binary tree. It is not. On our tree it is 4 2 5 1 3 6, which is nowhere near sorted. That property is a BST property, not a tree property.
- In “validate BST”, checking only `left->val < root->val < right->val`. That is a local check and it is wrong. Either do an inorder walk and confirm it is strictly increasing, or pass a valid (min, max) range downward.
- Losing the previous value while validating. You need to remember the previously printed node to compare against, usually a `TreeNode* prev` kept outside the recursion.

Postorder traversal

Meaning

Everything on the left, everything on the right, and only then the node itself. The root is always last.

Recursive algorithm

```
postorder(node):
    if node is empty: return
    postorder(node.left)
    postorder(node.right)
    record node
```

C++

```
void postorder(TreeNode* root, vector<int>& out) {
    if (root == NULL) return;
    postorder(root->left, out);
    postorder(root->right, out);
    out.push_back(root->val);
}
```

Dry run

```
post(1)
  post(2)
    post(4) -> record 4      out = [4]
    post(5) -> record 5      out = [4,5]
    record 2                  out = [4,5,2]
  post(3)
    left is NULL
    post(6) -> record 6      out = [4,5,2,6]
    record 3                  out = [4,5,2,6,3]
  record 1                    out = [4,5,2,6,3,1]
```

Notice the pattern: a node is recorded only after **both** of its subtrees are completely finished. The root is recorded last, always.

Complexity

Time $O(N)$, space $O(H)$.

When postorder is the right choice

This is the most important traversal for problem solving, and students usually underestimate it.

Postorder is the traversal for **bottom-up** information flow. Use it whenever the answer at a node depends on answers computed inside its subtrees:

- height of a tree
- number of nodes, number of leaves, sum of nodes
- diameter
- is the tree balanced
- maximum path sum
- deleting a tree (you must free the children before the parent, otherwise you lose the pointers)

- evaluating an expression tree: compute both operands, then apply the operator

Every one of those has the same skeleton:

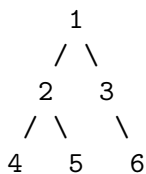
```
int solve(TreeNode* root) {
    if (root == NULL) return baseAnswer;
    int L = solve(root->left);
    int R = solve(root->right);
    return combine(L, R, root->val);    // this line is postorder position
}
```

If you understand that combine sits exactly where postorder records the node, you have understood Part 6 before we get there.

Common mistakes

- Recording the node before the recursive calls. That is preorder.
- Trying to build a postorder answer while descending. It cannot be done: at the moment you arrive at a node, you do not yet know anything about its subtrees.
- Writing the iterative version from memory without understanding it. That is Part 4, and it is the one traversal that genuinely needs care.

Comparing the three



Preorder	1 2 4 5 3 6	root first	top-down information
Inorder	4 2 5 1 3 6	root middle	BST and sorted order
Postorder	4 5 2 6 3 1	root last	bottom-up information

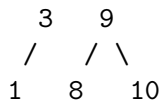
Facts worth remembering:

- In preorder, the **first** element is always the root.
- In postorder, the **last** element is always the root.
- Inorder alone cannot rebuild a tree, but inorder plus either of the other two can.
- All three visit every node once: $O(N)$ time, $O(H)$ space.
- Reversing a “root, right, left” preorder gives postorder. That trick is the basis of the easiest iterative postorder in Part 4.

A tiny exercise you should actually do

Take this tree:





Write down all three traversals before reading on.

Preorder : 7 3 1 9 8 10

Inorder : 1 3 7 8 9 10

Postorder : 1 3 8 10 9 7

The inorder came out sorted. That is not luck. This tree is a BST.

Check yourself

1. Which traversal gives you the root first, and why does that matter when copying a tree?
2. You are given the postorder array of a tree. Which element is the root, and what do you look for next?
3. Why is the space complexity $O(H)$ and not $O(N)$ for a balanced tree, even though there are N nodes?
4. A tree has inorder 4 2 5 1 3 6 and preorder 1 2 4 5 3 6. Which nodes are in the left subtree of the root, and how did you know?
5. You need to compute the height of every node. Which of the three traversals is the natural one, and why can it not be done in preorder?
6. What single line do you move to convert a preorder function into a postorder function?